UNIVERSITY OF LAYYAH

Department of Computer Science

ASSIGNMENT

Computer Organization and Assembly Language (COAL)

Topics

Sequential Execution | Branching Concept | Instruction Sequencing
Control Signals | Hardwired vs. Microprogrammed Control
Number Conversions (Binary and Decimal)

Name	Sara Manzoor
Roll No	UL-BSCSM-A-24-01
Department	Computer Science
University	University of Layyah
Submitted to	Mam Amna
Subject	COAL

1. Sequential Execution

Definition

Sequential execution is the process in which a computer executes instructions one after another in the order in which they appear in a program, unless a branching or control-transfer instruction changes the execution flow.

Explanation

The CPU uses the Program Counter (PC) to keep track of the address of the next instruction to be executed. In sequential execution, after fetching an instruction, the PC is normally updated to point to the next instruction in memory. The CPU generally performs the following steps:

1. Fetch: Retrieve the instruction from memory.
2. Decode: Interpret the instruction to determine the required operation.
3. Execute: Perform the operation.
4. Update PC: Move to the next instruction.

Example

Consider the following instructions:

```
MOV AX, 5
MOV BX, 10
ADD AX, BX
MOV CX, AX
```

The CPU executes these instructions in sequence:

- First, the value 5 is stored in register AX.
- Second, the value 10 is stored in register BX.
- Third, the values in AX and BX are added, producing 15 in AX.
- Finally, the value of AX is copied into register CX.

Result: AX = 15, BX = 10, and CX = 15.

Advantages

- Simple to understand and implement.
- Makes program execution predictable.
- Suitable for ordinary calculations and data-processing tasks.

Limitation

Sequential execution alone cannot efficiently handle decisions, loops, or situations that require jumping to another instruction.

2. Branching Concept

Definition

Branching is a control-flow mechanism that allows a program to change the normal sequence of instruction execution by transferring control to another instruction address.

Explanation

Branching is useful when a program must make decisions or repeat a set of instructions. Instead of always executing the next instruction, the CPU may jump to a different location. Branching instructions are commonly divided into two main types.

A. Unconditional Branching

An unconditional branch transfers control to a specified address regardless of any condition.

```
MOV AX, 5
JMP NEXT
MOV AX, 10

NEXT:
MOV BX, 20
```

The JMP NEXT instruction transfers control directly to the label NEXT. Therefore, MOV AX, 10 is skipped.

B. Conditional Branching

A conditional branch transfers control only when a specified condition is satisfied. These conditions are commonly determined using status flags set by previous instructions.

```
MOV AX, 5
CMP AX, 5
JE EQUAL

MOV BX, 0
JMP END

EQUAL:
MOV BX, 1

END:
```

The CMP instruction compares AX with 5. Since the values are equal, the Zero Flag is set, and JE EQUAL transfers control to the EQUAL label. Therefore, BX receives the value 1.

Applications of Branching

- Implementing if-else statements.
- Creating loops such as for and while.
- Handling errors and exceptional conditions.
- Implementing decision-making logic.

3. Instruction Sequencing

Definition

Instruction sequencing is the process of determining the order in which instructions are fetched and executed by the CPU.

Explanation

The CPU must determine which instruction should execute next. This process is controlled primarily by the Program Counter (PC), the control unit, and branch instructions. The normal instruction sequence consists of:

1. Fetch the instruction whose address is stored in the PC.
2. Decode the instruction.
3. Execute the required operation.
4. Update the PC to identify the next instruction.

In a simple fixed-length instruction architecture, the PC may be incremented by the instruction length. For variable-length instructions, it advances according to the decoded instruction length. A branch or jump may instead load a target address into the PC.

Example

Suppose the instruction addresses are 100, 104, 108, and 112. In normal sequential execution, the CPU may follow this sequence:

$$100 \rightarrow 104 \rightarrow 108 \rightarrow 112$$

If the instruction at address 104 branches to address 112, the sequence becomes:

$$100 \rightarrow 104 \rightarrow 112$$

The instruction at address 108 is skipped.

Factors Affecting Instruction Sequencing

- Program Counter updates.
- Conditional and unconditional branch instructions.
- Interrupts and exceptions.
- Instruction length and architecture.
- Pipeline hazards in pipelined processors.

Importance

Instruction sequencing ensures that instructions execute in the correct order and that decisions, loops, and function calls operate as intended.

4. Control Signals

Definition

Control signals are electrical or logical signals generated by the CPU's control unit to coordinate and control the activities of processor components.

Explanation

The control unit interprets each instruction and generates the signals required to perform it. These signals coordinate the ALU, registers, memory, and other components. For example, when the CPU executes an addition instruction, the control unit enables the appropriate registers, selects the addition operation in the ALU, and directs the result to the destination register.

Example: Executing an ADD Instruction

Consider the instruction ADD AX, BX. A simplified sequence of control actions is:

1. Enable AX and BX as source operands.
2. Select the addition operation in the ALU.
3. Route the ALU result to AX.
4. Enable AX to store the result.
5. Update the relevant status flags.

The exact signals and timing depend on the processor architecture.

Importance of Control Signals

- Coordinate CPU operations.
- Enable communication among components.
- Control memory access.
- Ensure instructions execute correctly.
- Maintain proper timing and synchronization.

5. Hardwired vs. Microprogrammed Control

The control unit generates the signals needed to execute machine instructions. Two major approaches to implementing a control unit are hardwired control and microprogrammed control.

A. Hardwired Control Unit

Definition: A hardwired control unit generates control signals using fixed hardware circuits, such as logic gates, decoders, and state machines.

Working: The instruction's opcode and the processor's current state are used as inputs to logic circuits. These circuits generate the control signals required for the operation.

Advantages

- Generally faster control-signal generation.
- Suitable for designs that prioritize high performance.
- Does not require a control memory containing microinstructions.

Disadvantages

- More difficult to modify after implementation.
- Complex instruction sets can make the control logic complicated.
- Design changes may require hardware redesign.

B. Microprogrammed Control Unit

Definition: A microprogrammed control unit generates control signals by executing microinstructions stored in a special control memory.

Working: The instruction is decoded, and the control unit selects a microprogram containing the sequence of microinstructions needed to carry out that machine instruction. Each microinstruction specifies the required control actions.

Advantages

- Easier to modify or extend the control sequence in many designs.
- Suitable for implementing complex instructions.
- Control behavior can be organized into microprograms.

Disadvantages

- May introduce additional control-memory access overhead.
- Can be slower than a comparable hardwired implementation, depending on design.
- Requires control memory and microinstruction sequencing hardware.

Conclusion

Sequential execution, branching, instruction sequencing, and control signals are fundamental concepts in computer organization. Sequential execution follows the normal instruction order, while branching changes that order when required. Instruction sequencing determines which instruction executes next, and control signals coordinate the CPU's internal operations. Hardwired and microprogrammed control units provide two different approaches to generating these signals. Understanding these concepts explains how a processor executes programs and manages its internal components efficiently.

6. Number Conversions: Binary and Decimal

Computers store and process data in binary, so converting between binary and decimal (including numbers with a fractional part) is an essential skill. The following examples show the method step by step.

Example 1: Binary to Decimal

Convert 111.000 (base 2) into decimal.

Step 1: To the left of the binary point the place values are 2^0, 2^1, 2^2, and to the right they are 2^{-1}, 2^{-2}, 2^{-3}.

Step 2: Multiply each digit by its power of 2.

$$= 1(2^2) + 1(2^1) + 1(2^0) + 0(2^{-1}) + 0(2^{-2}) + 0(2^{-3})$$

Step 3: Calculate.

$$= 4 + 2 + 1 + 0 + 0 + 0$$

111.000 (base 2) = 7.0 (base 10)

Example 2: Binary Fraction to Decimal

Convert 101.101 (base 2) into decimal.

$$= 1(2^2) + 0(2^1) + 1(2^0) + 1(2^{-1}) + 0(2^{-2}) + 1(2^{-3})$$

$$= 4 + 0 + 1 + 1/2 + 0 + 1/8$$

$$= 4 + 1 + 0.5 + 0.125$$

101.101 (base 2) = 5.625 (base 10)